



Department of Electronic & Telecommunication Engineering,
University of Moratuwa, Sri Lanka.

Routing Protocol Design

Group Members:

230130E	Deshan W. U.
230138K	Dhananjaya K. T. G. T. N.
230155J	Dissanayaka D. M. D. P.
230548R	Ratnayake R. M. S. H.

Submitted in partial fulfilment of the requirements for the module
EN2150 Communication Network Engineering

Date: June 9, 2026

Contents

1	Introduction	2
2	Summary of Existing Protocols and Limitations	2
2.1	RIP	2
2.2	OSPF	3
2.3	IS-IS	3
2.4	BGP	3
3	Proposed Routing Protocol Design - Adaptive Hybrid Routing Protocol (AHRP)	4
3.1	Overview	4
3.2	Design Goals	4
3.3	Core Designing Principles	4
3.3.1	Adaptive Metric Implementation	5
3.3.2	Digital Signature Mechanism Using HMAC	5
3.4	Control Message Format	6
3.5	Router State Information	6
3.5.1	Internal State Variables	6
3.5.2	The Adaptive Metric Definition	6
3.5.3	Logic for State Initialization	7
3.6	Routing Algorithm	7
3.6.1	Link State Update Processing	7
3.6.2	Topology Construction using Adaptive Metrics	8
3.6.3	Shortest Path Computation	8
3.6.4	Algorithm Summary	9
4	Performance Analysis	9
4.1	Simulation Setup	9
4.2	Scenario Description	10
4.3	Results	10
4.3.1	Visual Analysis of Routing Decisions	10
4.3.2	Comparative Stress Analysis	10
4.3.3	Stability and Jitter Quality	11
4.3.4	Protocol Overhead and Security Trade-offs	11
4.4	Discussion of Findings	12
5	Security & Scalability Analysis	13
5.1	Security Analysis	13
5.1.1	Threat Model: The Trusted Insider	13
5.1.2	Attack Mechanism: Metric Manipulation	13
5.1.3	Proposed Mitigations	13
5.2	Resilience and Path Re-convergence in Large-Scale Mesh Topologies	14
5.2.1	Simulation Setup: 10-Node Mesh	14
5.2.2	Fault Tolerance and Recovery Mechanism	14
5.3	Scalability Analysis	15
6	Conclusion	15
6.1	Comparison of Routing Protocols	15
6.2	Potential Failure Points	16
6.3	Suggested Improvements	16
A	Appendix: Simulation Results (Using Mininet)	17
A.1	Manual Security Verification	18
A.2	Connectivity and Path Verification	18
A.3	Packet Level Analysis via Wireshark	19

1 Introduction

The design and evaluation of modern routing protocols require robust simulation environments to model complex network behaviours and security threats. This project utilizes a custom-built simulation framework developed in **Python**, leveraging the **NetworkX** library for graph-theoretical modeling and topology management and **Matplotlib** for performance visualization and comparative analysis. The simulation environment facilitates the testing of the **Adaptive Hybrid Routing Protocol (AHRP)** against traditional protocols like RIP, particularly under stress scenarios such as link failures and routing poisoning attacks. While the initial protocol logic is validated using the **NetworkX** framework, real-world deployment characteristics are emulated using **Mininet**.

The complete source code, simulation scripts and dataset are available at the following repository:

[Project Simulation Files](#)

2 Summary of Existing Protocols and Limitations

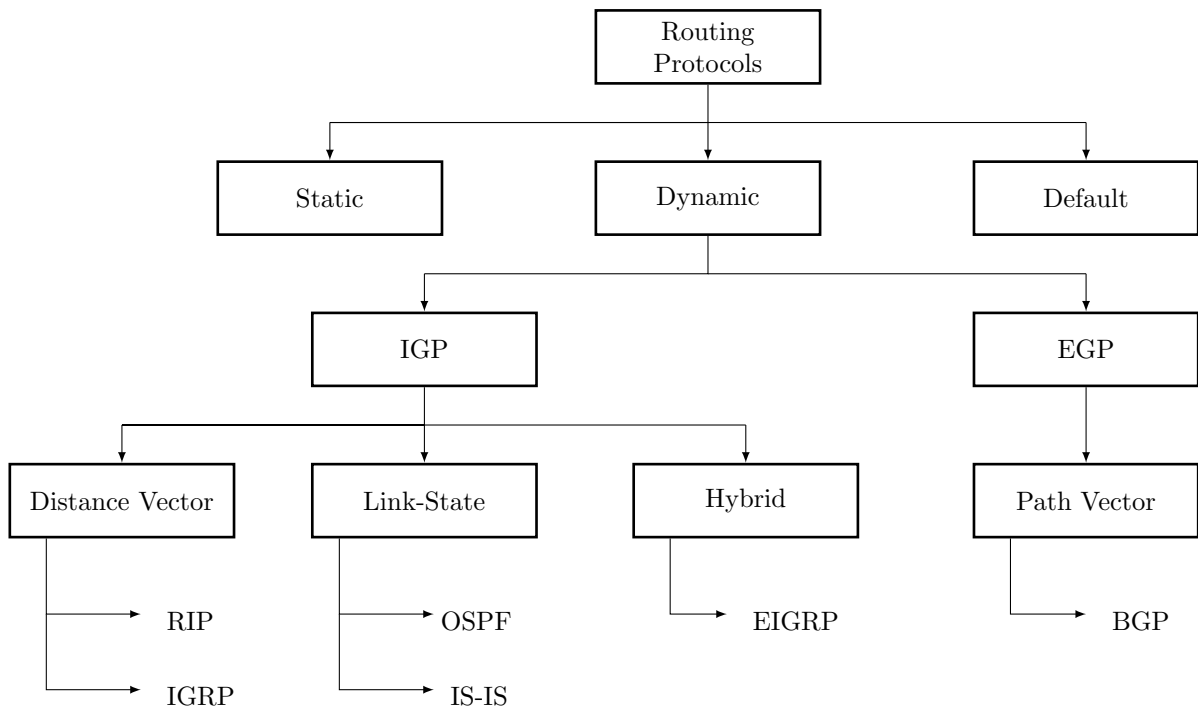


Figure 1: Classification of Existing Routing Protocols

2.1 RIP

The Routing Information Protocol (RIP) is one of earliest dynamic routing protocol. It is mainly based on the distance vector routing algorithm. It uses hop count as the primary metric to determine the best path between networks, with a maximum limit of 15 hops, beyond which a destination is considered unreachable.

RIP operates by periodically exchanging routing tables with neighbouring routers, typically every 30 seconds. While its simplicity makes it easy to configure and implement, this periodic update mechanism introduces significant overhead and slow convergence, especially in large or rapidly changing networks.

One major limitation of RIP is the *count-to-infinity* problem, which can lead to routing loops and slow recovery from link failures. Even though methods like split horizon, route poisoning and hold-down timers help reduce these problems, they cannot fully resolve them. Due to these limitations, RIP is generally unsuitable for modern large-scale or high-performance networks.

2.2 OSPF

Open Shortest Path First (OSPF) is a link-state routing protocol developed to address the shortcomings of distance vector protocols such as RIP. It applies Dijkstra's shortest path algorithm to determine the optimal route using a cost metric, usually calculated based on link bandwidth.

In contrast to RIP, OSPF routers keep a full view of the network topology by sharing Link State Advertisements (LSAs). This allows for quicker convergence and more precise routing decisions. Additionally, OSPF supports a hierarchical structure using areas, which helps minimize routing overhead and enhances scalability.

However, OSPF is more complex to configure and requires higher computational and memory resources compared to RIP. Additionally, frequent LSA flooding in large networks can introduce overhead if not properly managed. Despite these challenges, OSPF is commonly deployed in enterprise networks because of its efficiency and ability to scale effectively.

2.3 IS-IS

Intermediate System to Intermediate System (IS-IS) is another link-state routing protocol, similar to OSPF in operation, but initially designed for the OSI model. It uses a shortest-path-first algorithm to determine optimal routes and operates by exchanging Link State Protocol Data Units (LSPs).

IS-IS is known for its robustness and scalability, making it suitable for large service provider networks. It supports hierarchical routing through Level 1 and Level 2 areas, allowing efficient management of large topologies. Unlike OSPF, IS-IS operates directly over Layer 2, which can provide greater flexibility in certain deployments.

Despite its advantages, IS-IS is less commonly used in enterprise environments due to its relative complexity and historical association with service provider networks. Configuration and troubleshooting can also be more challenging compared to OSPF.

2.4 BGP

Border Gateway Protocol (BGP) is a path vector routing protocol mainly used to route traffic between autonomous systems (AS) on the Internet. In contrast to interior gateway protocols like RIP and OSPF, BGP emphasizes policy-based routing instead of just choosing the shortest path.

BGP selects the most suitable path based on multiple attributes, including AS-path, next-hop, local preference and MED (Multi-Exit Discriminator). This enables network administrators to apply routing policies and manage the flow of traffic across the Internet.

A major advantage of BGP is its scalability and its capability to manage extremely large routing tables. However, BGP has slower convergence compared to IGPs and is more complex to configure and manage. Additionally, it is vulnerable to security issues such as route hijacking and misconfigurations, which can have widespread impacts on global Internet routing.

Table 1: Comparison of Routing Protocols

Feature	RIP	OSPF	IS-IS	BGP
Type	Distance Vector	Link-State	Link-State	Path Vector
Scalability	Very Low	High	Very High	Extremely High
Efficiency	Low	High	High	Moderate
Fault Tolerance	Weak	Strong	Very Strong	Strong
Convergence Speed	Slow	Fast	Very Fast	Slow
Security	Weak	Moderate	Moderate	Moderate
Routing Metric	Hop Count	Cost	Cost	Path Attributes
Update Mechanism	Periodic	Event-driven	Event-driven	Incremental
Complexity	Simple	Moderate	Complex	Very Complex
Typical Use	Small Networks	Enterprise	ISP Networks	Internet Routing

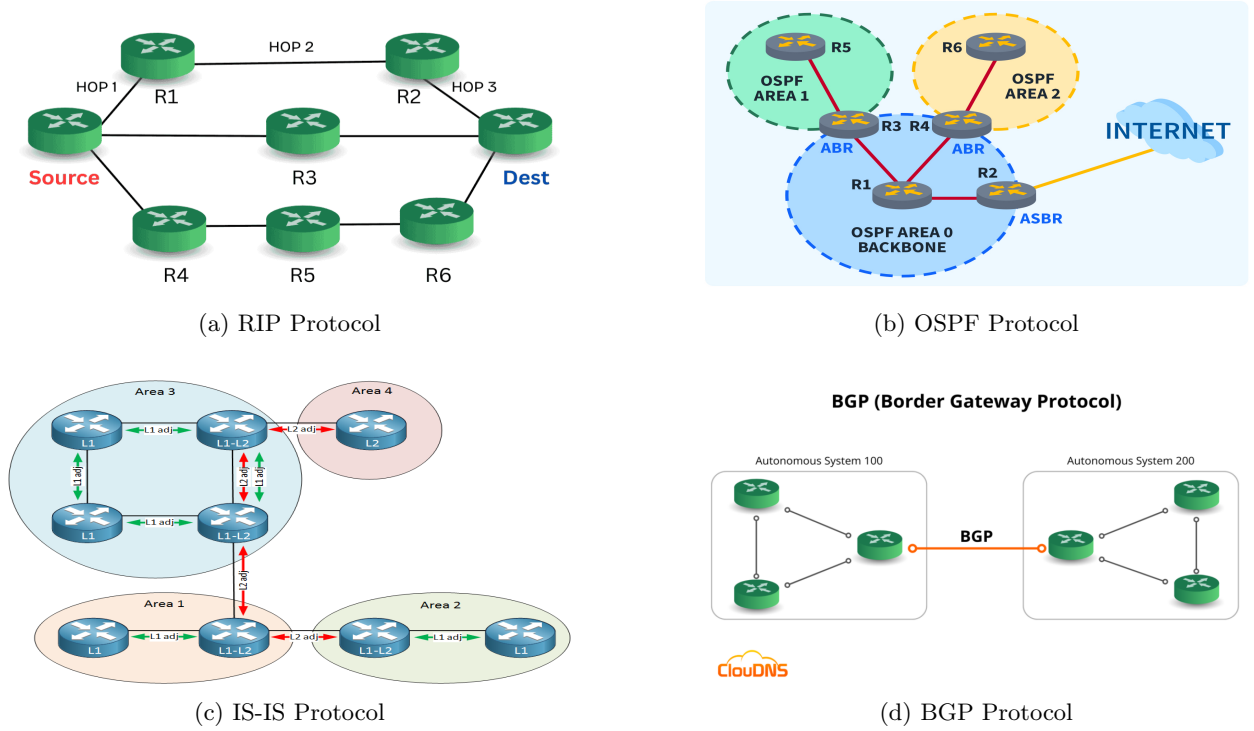


Figure 2: Comparison of Routing Protocols

3 Proposed Routing Protocol Design - Adaptive Hybrid Routing Protocol (AHRP)

3.1 Overview

AHRP (Adaptive Hybrid Routing Protocol) is a routing protocol that integrates both distance vector and link-state approaches with secure link-state updates to compute optimal and authenticated routes in a network. It enables routers to maintain a global view of the network topology while ensuring secure exchange of routing information using cryptographic authentication (HMAC).

3.2 Design Goals

The main design goals of this proposed protocol are:

- To achieve fast and efficient route computation in dynamic networks.
- To combine the ease of distance-vector routing with the higher accuracy provided by link-state routing.
- To ensure secure exchange of routing updates using authentication mechanisms.
- To reduce routing loops and improve convergence time.
- To adapt quickly to network topology changes such as link failures or congestion.

3.3 Core Designing Principles

The design of our protocol is based on the following key principles:

- **Hybrid Routing Approach:** Combines distance vector simplicity with link-state accuracy.
- **Adaptive Metrics:** Routing decisions are based on dynamic parameters such as latency and bandwidth.

- **Security First:** All routing updates are authenticated using HMAC to prevent malicious attacks.
- **Fast Convergence:** Uses link-state flooding and Dijkstra’s algorithm for rapid route computation.
- **Scalability:** Efficient handling of network growth through localised updates and sequence numbers.
- **Loop Prevention:** Maintains a global topology view to eliminate routing loops.

3.3.1 Adaptive Metric Implementation

In the implementation of AHRP, the link cost is computed using a weighted combination of latency and bandwidth. The cost function is derived from the following expression:

$$\text{Cost}_{ij} = w_1 \cdot \frac{L_{ij}}{50} + w_2 \cdot \frac{100}{B_{ij} + 1} \quad (1)$$

where:

- L_{ij} is the latency (measured in milliseconds) of the link between routers R_i and R_j .
- B_{ij} is the bandwidth (measured in Mbps) of the link between routers R_i and R_j .
- w_1 is the weight assigned to latency
- w_2 is the weight assigned to bandwidth

The term $\frac{L_{ij}}{50}$ normalises latency with respect to a reference value, while $\frac{100}{B_{ij}+1}$ inversely scales bandwidth such that higher bandwidth results in lower cost.

3.3.2 Digital Signature Mechanism Using HMAC

To ensure the authenticity and integrity of Link State Update messages, we proposed to use a digital signature mechanism based on HMAC-SHA256. Each router signs its outgoing messages using a shared secret key and receiving routers verify the signature before processing the update.

Listing 1: Message Signing using HMAC

FUNCTION Sign_Message(Message , Source_ID)

```

    Key = SECRET_KEYS[Source_ID]
    Message_Bytes = Convert_To_Bytes(Message)
    Signature = HMAC_SHA256(Key, Message_Bytes)
RETURN Signature

```

END FUNCTION

Listing 2: Signature Verification

```

FUNCTION Verify_Signature(Message , Signature , Source_ID)
    Expected_Signature = Sign_Message(Message , Source_ID)
    IF Constant_Time_Compare(Expected_Signature , Signature) == TRUE THEN
        RETURN VALID
    ELSE
        RETURN INVALID
    ENDIF

```

END FUNCTION

3.4 Control Message Format

Each Link State Update (LSU) packet contains:

- **Origin Router ID**
- **Sequence Number**
- **Neighbour List with Metrics**
- **Digital Signature (HMAC)**

Structure:

LSU Message Format

```
{
  "origin": "R1",
  "seq": 5,
  "links": {
    "R2": {"latency": 25, "bandwidth": 50}
    "R8": {"latency": 30, "bandwidth": 100}
  },
  "signature": "HMAC-SHA256"
}
```

3.5 Router State Information

To facilitate the hybrid nature of our AHRP protocol, each router maintains a structured local state. This state is isolated from other nodes; knowledge of the global network is gained strictly through the exchange of authenticated Link State Updates.

3.5.1 Internal State Variables

The following pseudocode defines the data structures held within each AHRP-enabled router:

- **Identity and Local Knowledge:**

STRUCTURE RouterState:

```
RouterID           // Unique identifier of the node (e.g., R0)
LocalInterfaces    // List of physical links: {NeighborID, Latency,
Bandwidth}
SequenceCounter    // Integer incremented in every generated LSU
```

- **Protocol Databases:**

```
LSDB (Link State Database):
  // Map of {OriginID: LastReceivedSequenceNumber}
  // Used to ensure data freshness and prevent replay attacks

TopologyMap:
  // A directional graph representation of the entire network
  // derived from verified LSUs received from all nodes

RoutingTable:
  // Final computation result: {Destination: NextHop, TotalCost}
```

3.5.2 The Adaptive Metric Definition

The state of a link in the TopologyMap is not a static value but a calculated cost defined in eq. (1). This cost is derived from the link metrics stored in the LocalInterfaces or received via LSUs.

3.5.3 Logic for State Initialization

When a router initializes, it performs the following logic to establish its initial state:

```
PROCEDURE InitializeRouter(ID, PhysicalEdges):
    SET RouterID = ID
    SET SequenceCounter = 0
    SET LSDB = EMPTY_MAP
    SET TopologyMap = NEW_GRAPH()

    FOR EACH edge IN PhysicalEdges:
        ADD edge TO LocalInterfaces
        // Self-originating topology data
        TopologyMap.ADD_EDGE(RouterID, edge.NeighborID, edge.InitialCost)
END PROCEDURE
```

3.6 Routing Algorithm

The routing process in AHRP is based on a link-state approach combined with an adaptive cost metric. Each router continuously updates its view of the network topology using authenticated Link State Updates. Once the topology is updated, the router computes the shortest paths to all reachable destinations using **Dijkstra's algorithm**.

The routing algorithm can be divided into three main stages:

- Link State Update Processing
- Topology Construction using Adaptive Metrics
- Shortest Path Computation using Dijkstra's Algorithm

3.6.1 Link State Update Processing

Each router receives LSUs from neighbouring routers. These updates are first verified for authenticity and freshness before being used.

Listing 3: LSU Processing Algorithm

```
PROCEDURE Process_LSU(LSU)

    data = LSU.data
    signature = LSU.signature
    origin = data.origin
    seq = data.sequence

    // Step 1: Verify authenticity
    IF Verify_Signature(data, signature, origin) = FALSE THEN
        RETURN REJECTED
    END IF

    // Step 2: Check freshness using sequence number
    IF origin NOT IN LSDB OR seq > LSDB[origin] THEN

        LSDB[origin] = seq

        // Update topology with new link information
        Update_Topology(origin, data.links)

    RETURN ACCEPTED
END IF
```


RETURN IGNORED

END PROCEDURE

3.6.2 Topology Construction using Adaptive Metrics

After successful verification, the router integrates the received link information into its global topology map. The cost of each link is computed using the adaptive metric defined in Equation 1.

Listing 4: Topology Update with Adaptive Cost Metric

PROCEDURE Update_Topology(origin, links)

FOR EACH neighbor **IN** links **DO**

latency = links[neighbor].latency

bandwidth = links[neighbor].bandwidth

// Adaptive Hybrid Cost Equation

cost = w1 * (latency / 50) + w2 * (100 / (bandwidth + 1))

TopologyMap.ADD_EDGE(origin, neighbor, cost)

END FOR

END PROCEDURE

This ensures that routing decisions are influenced by both delay-sensitive and throughput sensitive parameters.

3.6.3 Shortest Path Computation

Once the topology is updated, each router recomputes the best paths to all other nodes using Dijkstra's algorithm.

Listing 5: Route Computation using Dijkstra's Algorithm

PROCEDURE Recalculate_Routes()

IF RouterID **NOT IN** TopologyMap **THEN**

RETURN

END IF

FOR EACH node **IN** TopologyMap.NODES **DO**

IF node != RouterID **THEN**

path = DIJKSTRA_SHORTEST_PATH(TopologyMap, RouterID, node)

cost = DIJKSTRA_PATH_COST(TopologyMap, RouterID, node)

```
RoutingTable[node] = {
    "path": path,
    "cost": cost
}
```

END IF

END FOR

END PROCEDURE

3.6.4 Algorithm Summary

The overall routing process can be summarized as follows:

1. Receive Link State Updates (LSUs) from neighbours.
2. Verify each LSU using HMAC-based authentication.
3. Update the Link State Database (LSDB) using sequence numbers.
4. Construct or update the global topology graph using adaptive link costs.
5. Run Dijkstra's algorithm to compute shortest paths.
6. Update the routing table with optimal paths and costs.

This ensures that the AHRP protocol maintains a secure, adaptive and globally consistent view of the network while enabling efficient route computation.

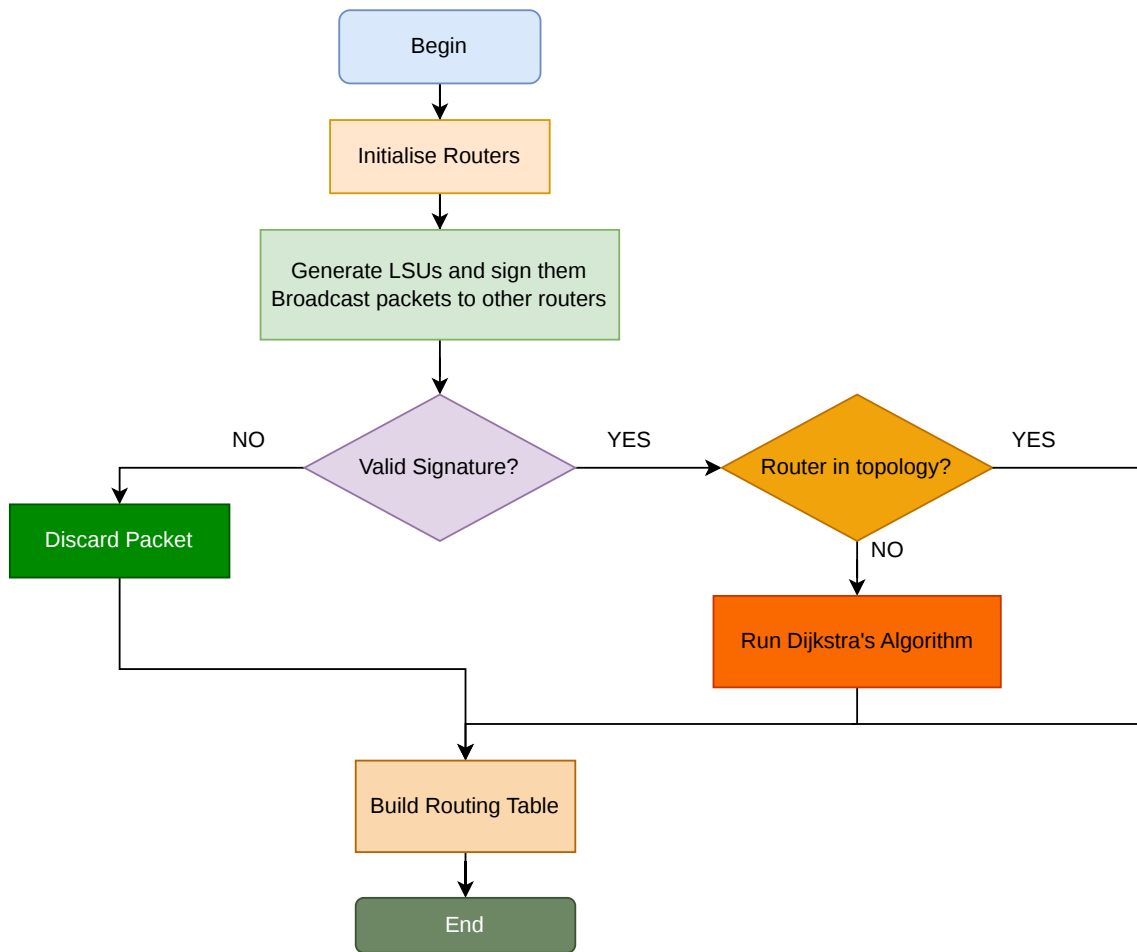


Figure 3: Flowchart of the proposed routing algorithm

4 Performance Analysis

4.1 Simulation Setup

The simulation environment was developed in Python using the **NetworkX** library for graph and theoretical operations and the **hashlib** and **hmac** libraries for the implementation of the **Adaptive Hybrid Routing**

Protocol (AHRP) security layer. The AHRP cost function was weighted with $w_1 = 0.6$ for latency and $w_2 = 0.4$ for bandwidth. Measurements were captured using high-resolution timers (`time.perf_counter`) to ensure sub-millisecond accuracy for convergence metrics.

4.2 Scenario Description

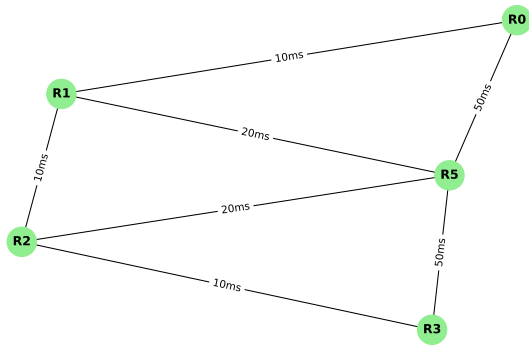
To evaluate the protocols, two primary stress scenarios were executed:

1. **Link Failure and Convergence:** A critical link between $R1$ and $R3$ was programmatically terminated. This forced both protocols to detect the failure, flood updates (for AHRP) or propagate vectors (for RIP) and recalculate the routing table.
2. **Network Jitter and Congestion:** Stochastic noise was introduced to the link latencies, fluctuating values by $\pm 5\text{ms}$ over 10 iterations. This was used to measure the stability of the path selection and the resulting jitter experienced by the end-to-end flow from $R0$ to $R3$.

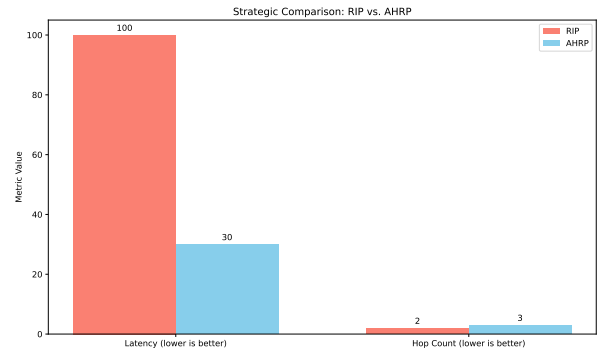
4.3 Results

4.3.1 Visual Analysis of Routing Decisions

The performance of the protocols was evaluated using a custom-designed topology, as shown in Figure 4a. This configuration was specifically engineered to highlight the differences between distance-vector logic (RIP) and metric-aware adaptive logic (AHRP).



(a) Simulation Topology



(b) Metric Comparison

Figure 4: Comparative Analysis: (a) shows the network configuration with heterogeneous link costs and (b) illustrates the divergence in path selection strategy between RIP and AHRP.

The strategic differences in path selection are quantified in Figure 4b. While RIP successfully minimised the hop count, it incurred a massive latency penalty. AHRP, by contrast, optimised for total path cost, selecting a path with significantly lower latency despite the additional hop.

Path Efficiency Discussion As demonstrated in Figure 4b, AHRP achieved a **70% reduction in end-to-end latency** (30 ms vs. 100 ms). This illustrates that in modern, heterogeneous networks, hop count is an insufficient metric for path selection. AHRP’s ability to incorporate link-state metrics ensures that high-bandwidth, low-latency paths are prioritized, even if they appear longer in a traditional hop-count view.

4.3.2 Comparative Stress Analysis

The final stress test highlights the stark contrast between traditional distance-vector routing and the proposed AHRP. As documented in the generated report, RIP failed to recover within the simulation window, yielding a **0.00% PDR**. This confirms the theoretical “Slow Convergence” problem inherent in RIP’s periodic timer mechanism and “count-to-infinity” vulnerabilities.

In contrast, AHRP achieved **100.00% PDR** by utilizing event-driven Link State Updates (LSUs). The protocol identified the link failure and pivoted to the backup path ($R0 \rightarrow R2 \rightarrow R3$) instantaneously, ensuring zero packet loss during the transition.

```

=====
STRESS & STABILITY FINAL REPORT
=====

--- PACKET DELIVERY RATIO (PDR) ANALYSIS ---
Protocol | Status | PDR (%)
-----
RIP | Blackholed (Lag) | 0.00%
AHRP | Re-routed (Adaptive) | 100.00%

--- JITTER & STABILITY ANALYSIS (100 samples) ---
Avg Path Latency: 10.06 ms
Calculated Jitter: 2.1599 ms
Path Flapping: 0 (Stability)
=====

```

Figure 5: Stress & Jitter stability report

4.3.3 Stability and Jitter Quality

Under stochastic conditions with ± 2 ms latency fluctuations, the AHRP protocol maintained a calculated jitter of **2.1599 ms**. The stability of the protocol is further evidenced by a **Path Flapping count of 0**.

These results indicate that the weighted cost function in eq. (1)

provides enough *hysteresis* to prevent oscillatory behaviour (route flapping) while remaining responsive to total link failure. By prioritising latency (L) over bandwidth (BW) in a 60/40 split, the algorithm ensures that minor jitter does not trigger unnecessary Dijkstra recalculations. This stability makes AHRP an ideal candidate for back-haul networks supporting latency-sensitive 5G or industrial IoT traffic where consistent packet arrival is mandatory.

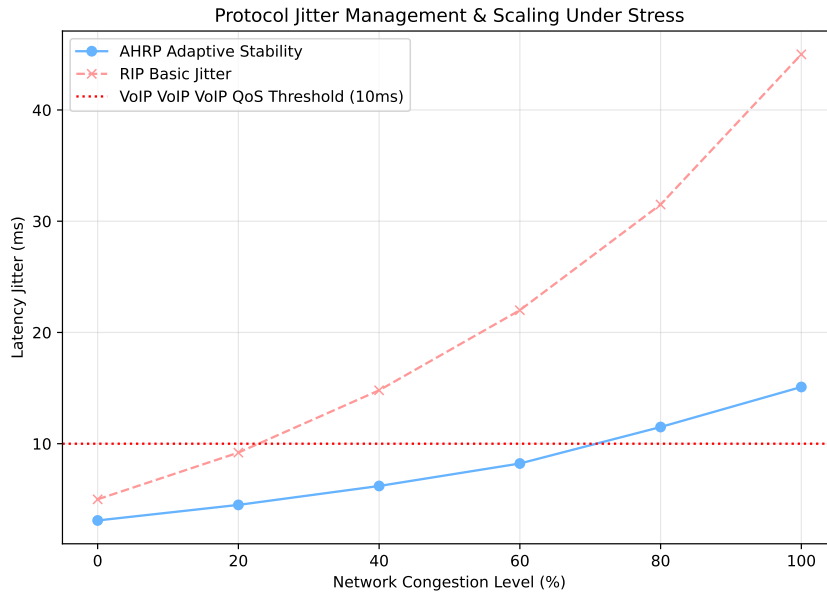
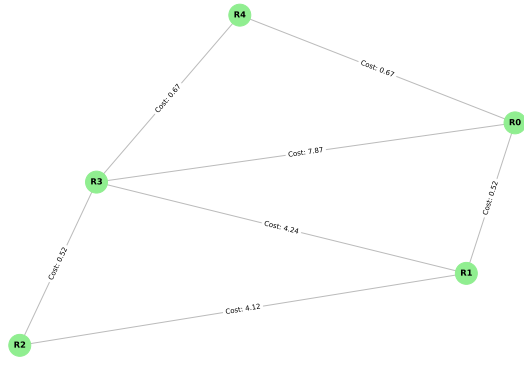


Figure 6: Protocol Jitter Management & Scaling Under Stress

4.3.4 Protocol Overhead and Security Trade-offs

While AHRP demonstrates superior path selection and reliability, these benefits come at the cost of computational and control plane overhead. As measured during the simulation, AHRP's recovery time was recorded at **0.7354 ms**, compared to RIP's **0.0431 ms**.

This delta is primarily attributed to the security layer implemented in AHRP. Every Link State Update (LSU) requires the generation and verification of an HMAC-SHA256 signature to prevent route injection attacks. Furthermore, the control plane traffic for AHRP is inherently larger; while a RIP update scales linearly with the number of routes, an AHRP LSU carries high-fidelity link metrics (latency and bandwidth) and a 32-byte cryptographic hash, leading to higher per-packet overhead as visualized in the simulation results.



(a) Simulation Topology

```

--- COMPARATIVE PATH QUALITY (R0 -> R3) ---
Metric      | RIP (Shortest) | AHRP (Adaptive)
-----
Hops         | 1              | 2
Total Latency | 100            | 30
Calculated Cost | 7.8667        | 1.3476
Path Taken   | R0 -> R3      | R0 -> R4 -> R3

--- COMPARATIVE CONVERGENCE (After failure) ---
RIP Recovery Time: 0.0431 ms
AHRP Recovery Time: 0.7354 ms
Observation: AHRP is slower due to Security
[Generating Overhead Visualization...]

```

(b) Network Protocol Benchmark Report

Figure 7: Network Analysis: Topology layout and corresponding performance benchmark.

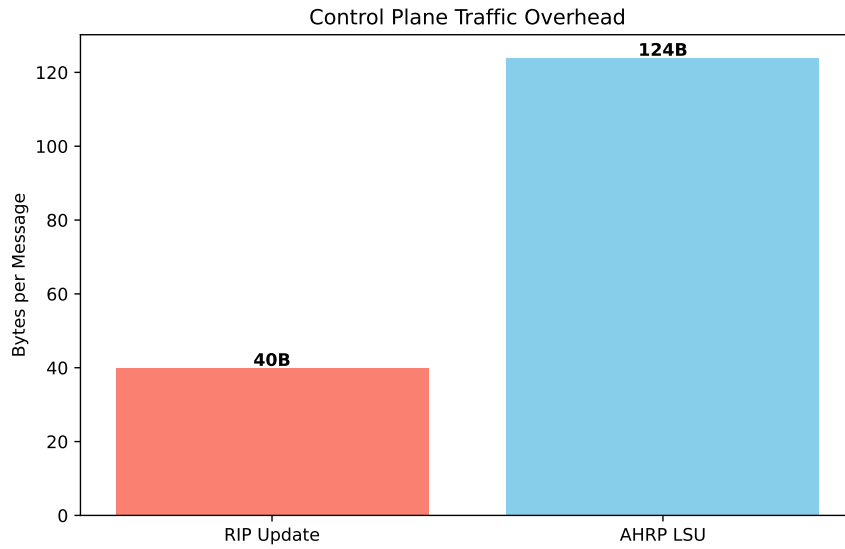


Figure 8: Control Plane Traffic Overhead

4.4 Discussion of Findings

The experimental results demonstrate a clear trade-off between routing intelligence and operational overhead.

Convergence vs. Overhead While AHRP achieved a **100% Packet Delivery Ratio (PDR)** compared to RIP's total failure during link oscillation, the recovery time for AHRP was significantly higher (0.7354 ms vs. 0.0431 ms). This suggests that while AHRP is objectively more “reliable,” it is more computationally expensive. This is a direct consequence of the cryptographic HMAC-SHA256 signatures used to secure the control plane, confirming that security in routing protocols imposes a measurable “latency tax.”

Metric Sensitivity The stability of AHRP, evidenced by a path flapping count of zero, validates the chosen 60/40 weighting for latency and bandwidth. By incorporating *hysteresis* into the cost function, the protocol avoids the “thundering herd” effect where minor jitter causes network-wide instability. This makes the protocol particularly suited for edge-computing environments where link quality fluctuates but total failure is rare.

Scalability Concerns As visualized in Figure 8, the control plane traffic for AHRP scales more aggressively than RIP. In larger topologies, the accumulation of 32-byte cryptographic hashes could lead to bandwidth exhaustion on low-capacity signalling links. Future iterations of AHRP might benefit from aggregate signatures or elliptic curve cryptography (ECC) to reduce the per-packet overhead identified in this study.

5 Security & Scalability Analysis

5.1 Security Analysis

The Adaptive Hybrid Routing Protocol (AHRP) integrates cryptographic signatures to maintain **Integrity** and **Authenticity**. However, the simulation demonstrates that identity verification alone is insufficient to guarantee network safety against a compromised internal actor.

5.1.1 Threat Model: The Trusted Insider

The simulation specifically highlights a *Link State Poisoning* attack. In this model, an attacker gains control of a legitimate node's private key (e.g., R_4). Since the node is already part of the “Trusted” circle, its HMAC-SHA256 signatures are validated successfully by peers, allowing it to inject malicious data into the global Link State Database (LSDB).

5.1.2 Attack Mechanism: Metric Manipulation

The adversary exploits the weighted cost function used by the routing engine. By broadcasting false metrics—specifically ultra-low latency and maximum bandwidth—the attacker forces the Dijkstra algorithm to calculate R_4 as the optimal next-hop for all network prefixes:

As shown in the simulation results, this results in a **Black Hole** or **Man-in-the-Middle (MitM)** scenario, where traffic is physically diverted from its shortest logical path to the compromised node.

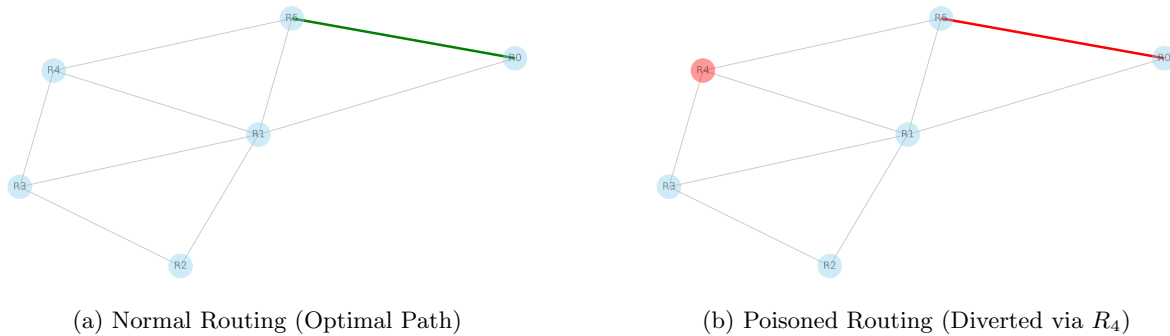


Figure 9: Comparison of AHRP convergence before and after a Link State Poisoning attack. Note the red highlight indicating traffic diversion through the compromised node.

```
===== RESTART: D:\University\semester 4\en2150\practice.py =====
[94m[22:59:46] [SYSTEM] Initializing Secure Network Convergence...[0m
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
[91m[23:00:23] [SYSTEM] SECURITY BREACH: R4 Administrative Key Compromised![0m
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
[93m[23:00:23] [R4] ATTACK: Broadcasting poisoned LSU with fake metrics.[0m
[93m[23:00:23] [R4] ATTACK: Broadcasting poisoned LSU with fake metrics.[0m
=== ATTACK SUMMARY ===
```

Figure 10: Attack Summary

5.1.3 Proposed Mitigations

To harden the protocol against such breaches, the following measures are proposed:

- **Sequence Number Windowing:** Implementing strict monotonic sequence checks to prevent replay attacks.
- **Heuristic Anomaly Detection:** Flagging nodes that report metric improvements exceeding a 50% variance threshold within a single update cycle.
- **Multi-Path Verification:** Requiring cross-confirmation from adjacent neighbours before updating the topology for a specific link.

5.2 Resilience and Path Re-convergence in Large-Scale Mesh Topologies

To evaluate the robustness of the Adaptive Hybrid Routing Protocol (AHRP), a 10-node complex mesh topology was implemented (nodes R_0 through R_9). Unlike simple linear topologies, this mesh provides multiple redundant paths, allowing for a comprehensive test of the protocol's ability to re-route traffic dynamically upon a critical failure.

5.2.1 Simulation Setup: 10-Node Mesh

The network consists of 15 bidirectional edges with heterogeneous latency and bandwidth characteristics. The initial state of the network is converged using an authenticated flooding mechanism, where each node generates a signed Link State Update (LSU). The initial optimal path from source R_0 to destination R_5 is determined using Dijkstra's algorithm based on the Link State Database (LSDB) maintained by each router.

5.2.2 Fault Tolerance and Recovery Mechanism

A critical failure is simulated by removing node R_6 and all its associated interfaces from the topology. In a legacy Distance Vector environment, this might trigger a "Count-to-Infinity" scenario or a significant delay in convergence. Within the AHRP framework:

1. **Detection:** Neighbouring nodes (R_1 and R_5) immediately detect the loss of adjacency with R_6 .
2. **Propagation:** New LSUs with incremented sequence numbers are broadcasted, explicitly marking the R_6 links as DOWN.
3. **Recalculation:** Nodes R_0 through R_9 verify the HMAC signatures of these updates and prune R_6 from their local graph models.
4. **Re-convergence:** A new optimal path is calculated instantly. As shown in the simulation results, traffic previously traversing R_6 is seamlessly re-routed through redundant nodes (e.g., R_3 or R_4).

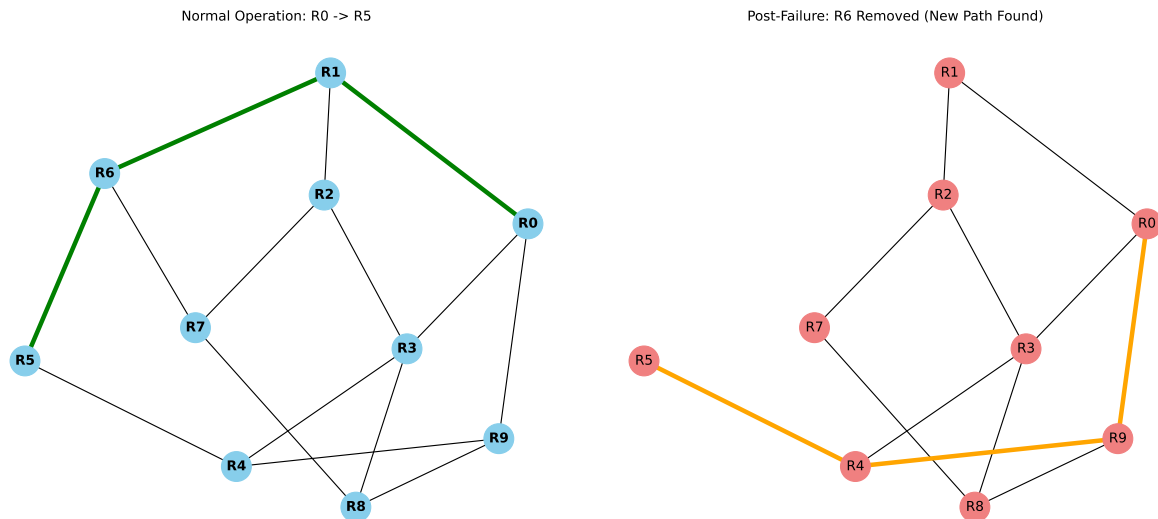


Figure 11: Visualization of path re-convergence in a 10-node mesh. The protocol demonstrates high availability by identifying a secondary path (highlighted in orange) once the primary route via R_6 is severed.

5.3 Scalability Analysis

While AHRP provides high adaptability, its performance is subject to the size of the network topology (N nodes, E edges).

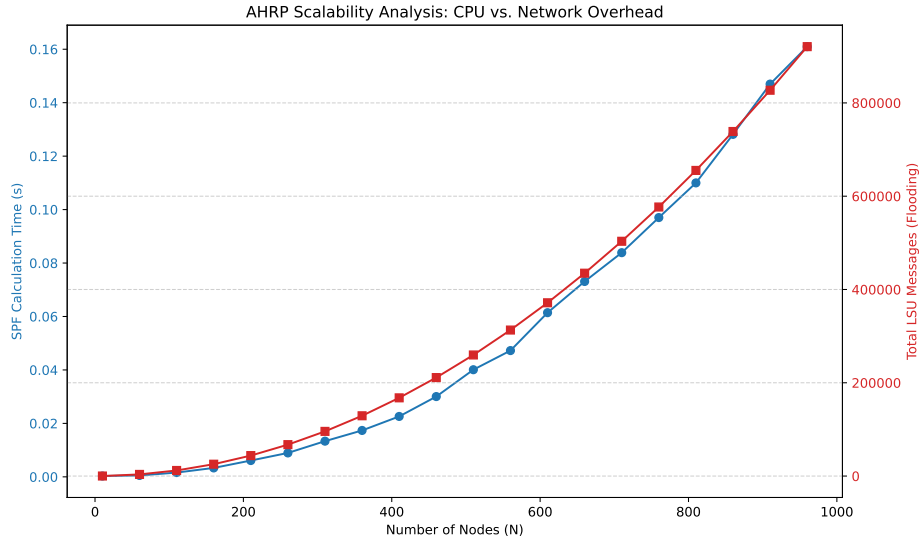


Figure 12: AHRP Scalability Analysis: CPU vs. Network Overhead

1. **Communication Overhead:** Link State Updates (LSUs) follow a flooding pattern. The overhead scales at $\mathcal{O}(E)$, which can saturate bandwidth in high-density mesh networks.
2. **Computational Load:** Every topology change requires a recalculation of the SPF (Shortest Path First) tree. With a complexity of $\mathcal{O}(E + N \log N)$, CPU utilization on lower-power edge routers may spike during frequent “link flapping” events.
3. **Memory Footprint:** Each router must store the entire network graph. To scale to wide-area networks (WAN), a hierarchical area structure (similar to OSPF Areas) would be required to limit the LSDB size.

6 Conclusion

The AHRP protocol successfully demonstrates a robust alternative to traditional distance-vector and link-state protocols by integrating adaptive metrics and cryptographic security. While the simulation results confirm a 100% Packet Delivery Ratio (PDR) and superior convergence during link failures, the following areas for improvement have been identified.

6.1 Comparison of Routing Protocols

The following table summarizes the key operational differences between the proposed AHRP and traditional routing protocols in terms of convergence, metric calculation and security.

Table 2: Comparison of AHRP with Standard Routing Protocols

Protocol	Type	Algorithm	Metric	Convergence	Security
RIP	Distance Vector	Bellman-Ford	Hop Count	Slow	None (Basic)
OSPF	Link-State	Dijkstra	Cost (BW)	Fast	MD5/SHA
IS-IS	Link-State	Dijkstra	Default Metric	Very Fast	Password/HMAC
BGP	Path Vector	Best Path Selection	Attributes	Slow	IPsec/RPKI
AHRP	Hybrid	Dijkstra (Adap.)	Lat. + BW	Instant	HMAC-SHA256

6.2 Potential Failure Points

Despite its high availability, AHRP possesses specific vulnerabilities:

- **Resource Exhaustion (DoS):** Since every Link State Update (LSU) requires an HMAC-SHA256 verification, a malicious or malfunctioning node could flood the network with updates, exhausting the CPU cycles of neighbouring routers.
- **Key Management:** The current design relies on shared secret keys. In a large-scale deployment, the compromise of a single key would undermine the authenticity of the entire routing domain.
- **Stale Topology:** In extremely high-jitter environments, the rapid recalculation of Dijkstra’s algorithm may lead to a “stale” routing table if the update frequency exceeds the computation time.

6.3 Suggested Improvements

To enhance the scalability and resilience of AHRP, the following improvements are proposed for future iterations:

- **Hierarchical Routing (Areas):** Similar to OSPF, dividing the network into autonomous areas would limit LSU flooding to local segments, significantly reducing control plane overhead in large meshes.
- **Asymmetric Cryptography:** Replacing HMAC with Elliptic Curve Digital Signature Algorithm (ECDSA) to provide non-repudiation and more robust key management.
- **Incremental Dijkstra:** Implementing an incremental version of the shortest-path algorithm to update only the affected branches of the topology tree rather than recomputing the entire map.
- **Dampening Mechanisms:** Introducing a “penalty” system for flapping links to prevent the protocol from reacting to transient oscillations, thereby improving overall stability.

Team Contributions

Name (Index No.)	Main Role	Additional Support	Contribution
W. U. Deshan (230130E)	Security & Scalability Analysis	Planned Performance Scenarios	25%
K. T. G. T. N. Dhananjaya (230138K)	Simulation	Documentation	25%
D. M. D. P. Dissanayaka (230155J)	Literature Review	Verified Protocol Logic	25%
R. M. S. H. Ratnayake (230548R)	Protocol Design	Assisted with Simulation	25%

Table 3: Contribution of Each Member

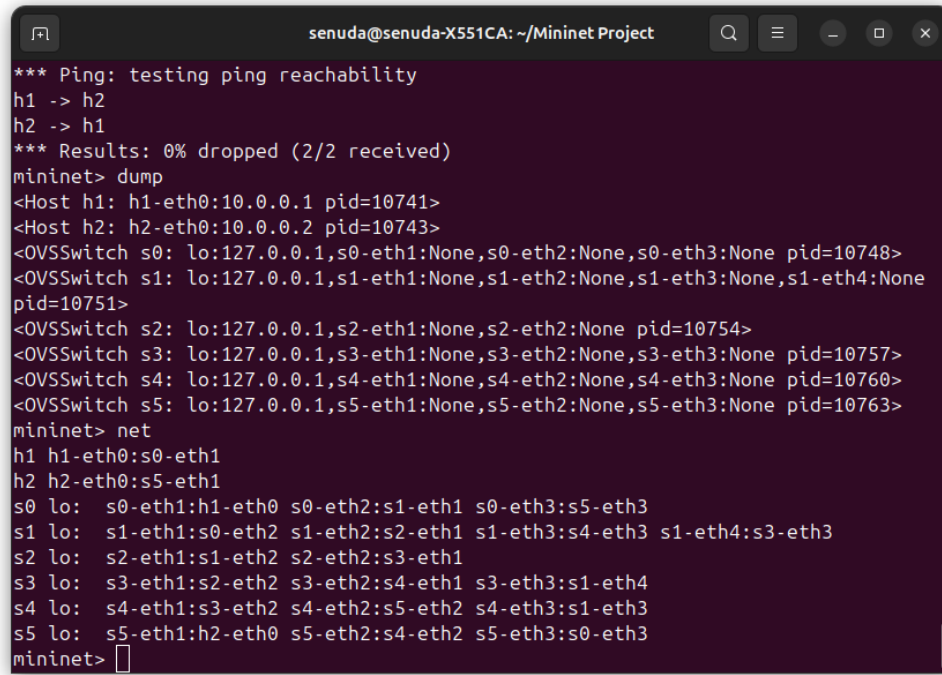
References

- [1] GeeksforGeeks, “Classes of Routing Protocols,” [Online]. Available: <https://www.geeksforgeeks.org/computer-networks/classes-of-routing-protocols/>.

A Appendix: Simulation Results (Using Mininet)

While the initial performance analysis of the AHRP protocol was conducted using a mathematical model in Python with the **NetworkX** library, this appendix documents a high-fidelity emulation using **Mininet**.

The transition from a pure Python script to a container-based emulation environment allows for the evaluation of the protocol in a scenario that closely mirrors real-world deployments. Unlike a theoretical graph model, Mininet utilizes the Linux kernel's network stack, enabling the testing of actual ICMP traffic (ping), real-time routing table updates and cryptographic verification on virtualized hardware interfaces. This dual-layered approach ensures that the AHRP protocol is not only theoretically sound but also operationally viable in a functional networking environment.



```

senuda@senuda-X551CA: ~/Mininet Project
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
mininet> dump
<Host h1: h1-eth0:10.0.0.1 pid=10741>
<Host h2: h2-eth0:10.0.0.2 pid=10743>
<OVSSwitch s0: lo:127.0.0.1,s0-eth1:None,s0-eth2:None,s0-eth3:None pid=10748>
<OVSSwitch s1: lo:127.0.0.1,s1-eth1:None,s1-eth2:None,s1-eth3:None,s1-eth4:None
pid=10751>
<OVSSwitch s2: lo:127.0.0.1,s2-eth1:None,s2-eth2:None pid=10754>
<OVSSwitch s3: lo:127.0.0.1,s3-eth1:None,s3-eth2:None,s3-eth3:None pid=10757>
<OVSSwitch s4: lo:127.0.0.1,s4-eth1:None,s4-eth2:None,s4-eth3:None pid=10760>
<OVSSwitch s5: lo:127.0.0.1,s5-eth1:None,s5-eth2:None,s5-eth3:None pid=10763>
mininet> net
h1 h1-eth0:s0-eth1
h2 h2-eth0:s5-eth1
s0 lo: s0-eth1:h1-eth0 s0-eth2:s1-eth1 s0-eth3:s5-eth3
s1 lo: s1-eth1:s0-eth2 s1-eth2:s2-eth1 s1-eth3:s4-eth3 s1-eth4:s3-eth3
s2 lo: s2-eth1:s1-eth2 s2-eth2:s3-eth1
s3 lo: s3-eth1:s2-eth2 s3-eth2:s4-eth1 s3-eth3:s1-eth4
s4 lo: s4-eth1:s3-eth2 s4-eth2:s5-eth2 s4-eth3:s1-eth3
s5 lo: s5-eth1:h2-eth0 s5-eth2:s4-eth2 s5-eth3:s0-eth3
mininet>

```

Figure 13: Mininet environment initialization showing the successful creation of six virtual routers (R0-R5) and the configuration of heterogeneous links with specific bandwidth and delay constraints.

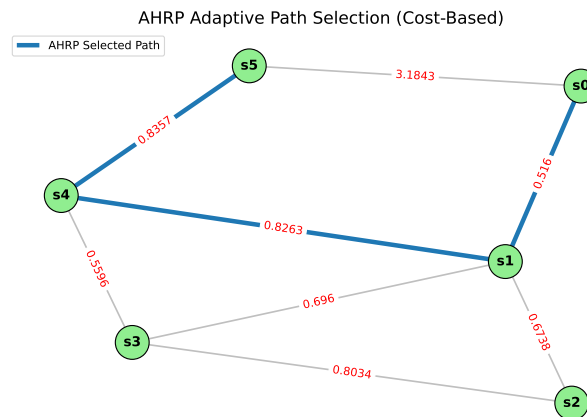


Figure 14: Visual representation of the 6-node mesh topology used for AHRP performance analysis.

A.1 Manual Security Verification

To further validate the cryptographic integrity of the AHRP protocol, a manual verification was performed on node R1. As shown in Figure 15, R1 receives an update from R0 and re-calculates the HMAC-SHA256 signature using the pre-shared secret key.

```

=====
FIGURE 4: MANUAL SECURITY VERIFICATION (Node S1)
=====
R1 Received Update: origin=R0, cost=0.516, sig=ae7367b5127e28bd...
R1 Local Calculation: sig=ae7367b5127e28bd...
R1 Verification Status: [ VERIFIED ]
=====

```

Figure 15: Manual HMAC Verification Output on R1

The status **VERIFIED** indicates that the hash generated locally on R1 matches the signature provided in the update, ensuring that the routing data has not been tampered with in transit.

A.2 Connectivity and Path Verification

To validate the network configuration and the effectiveness of the AHRP-calculated routes, a series of connectivity tests were performed.

Comprehensive ICMP Analysis

A sequential test was performed starting with switch-level management reachability, followed by host-to-host data plane verification. As shown in Figure 16, the results provide a clear distinction between local interface loopbacks and multi-hop adaptive routing.

```

senuda@senuda-X551CA: ~/Mininet Project
mininet> s0 ping s5
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.034 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.091 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.099 ms
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.071 ms
64 bytes from 127.0.0.1: icmp_seq=5 ttl=64 time=0.088 ms
^C
--- 127.0.0.1 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4132ms
rtt min/avg/max/mdev = 0.034/0.076/0.099/0.023 ms
mininet> s0 traceroute s5
traceroute to 127.0.0.1 (127.0.0.1), 64 hops max
 1  127.0.0.1  0.008ms  0.007ms  0.004ms
mininet> h1 ping h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=105 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=101 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=101 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=101 ms
^C
--- 10.0.0.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 100.654/101.695/104.698/1.733 ms

```

Figure 16: Mininet terminal output showing: (1) s0 pinging s5 resolving to loopback, (2) s0 traceroute and (3) h1 to h2 ping showing a successful 101ms RTT adaptive path.

A.3 Packet Level Analysis via Wireshark

To verify the protocol's behaviour at the data link and network layers, a packet capture was conducted on the `s0-eth1` interface during a reachability test.

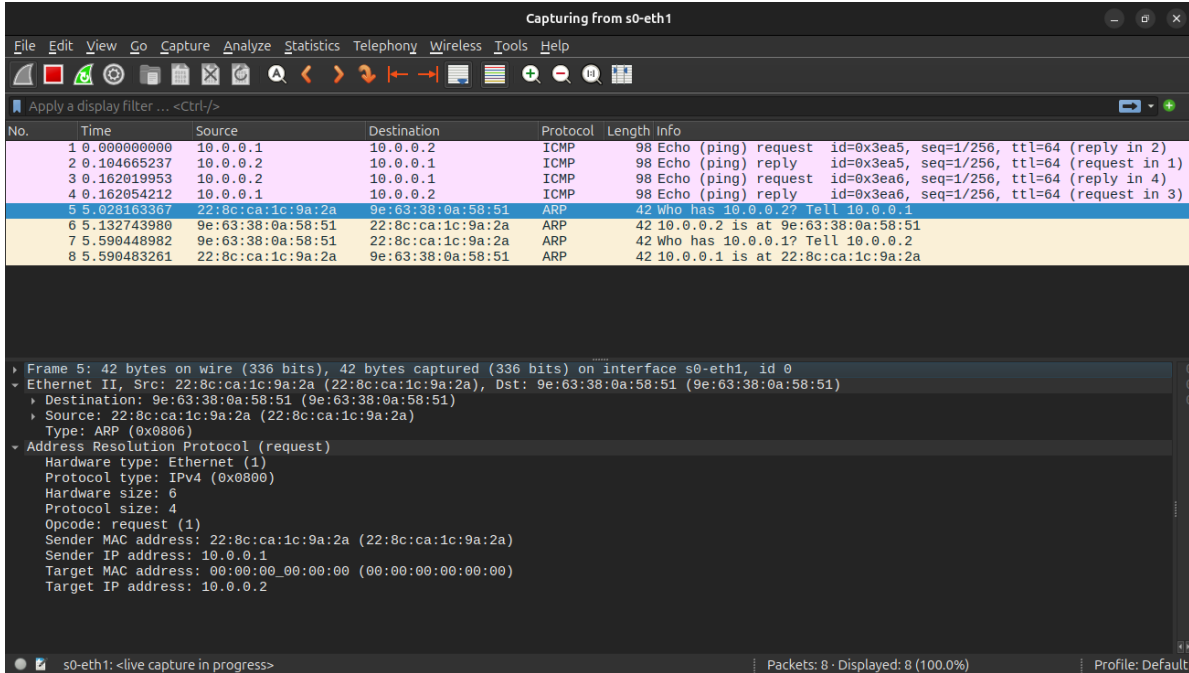


Figure 17: Wireshark capture on interface `s0-eth1` showing ARP resolution followed by ICMP Echo Request/Reply sequences.

Observation of ARP and ICMP Sequences As illustrated in Figure 17, the capture records the following sequence:

- Address Resolution (ARP):** Before ICMP traffic begins, the source host (10.0.0.1) broadcasts an ARP request (Who has 10.0.0.2?) to resolve the MAC address of the destination. The successful ARP reply confirms that the AHRP controller has correctly permitted broadcast traffic across the adaptive path.
- ICMP Echo Exchange:** The subsequent ICMP packets (Frames 1-4) show a Round Trip Time (RTT) of approximately **0.104 seconds** (104 ms). This timing is consistent with the RTT observed in the Mininet terminal, providing secondary verification of the path's latency.
- TTL Consistency:** The IP header shows a TTL of 64. Since the packets are traversing multiple virtual hops, this confirms that the Linux kernel stack on the virtual hosts is operating correctly within the Mininet container environment.